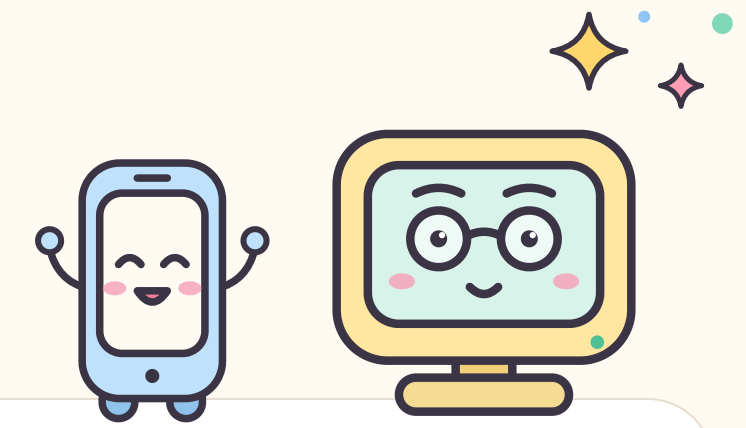




第 15 集 · 约 27 分钟

测试服务端

“我们加密了”只是一句话，证据在测试里

15

从零写出 LanShare

这一集你会学到

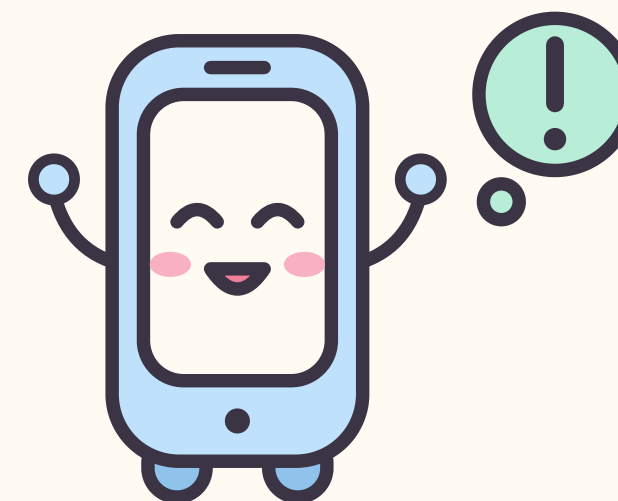
1. 集成测试：tests 目录下每个文件都是一个独立的 crate，只能用公开的接口
2. 测试脚手架：随机端口真实启动、可调的限额、假的桌面、收集日志
3. 按协议说话的测试客户端：既能当守规矩的小手机，也能当捣蛋鬼
4. 抓包测试：录下线路上的每一个字节，证明里面没有秘密
5. 回归测试和变异测试：每条审查发现一个测试，把修复改回去，测试必须变红



先想一个问题

怎么证明“同一个 WiFi 下的窃听者，什么都看不到”？

在客户端和服务端之间夹一个代理，录下每一个字节。完整走一遍配对、上传、下载、文字，再到录音里搜秘密。





单元测试和集成测试

● 单元测试写在源文件里的 tests 模块，能测私有函数

前面几集写的 23 个都是这种

● 集成测试放在 crate 根目录的 tests 文件夹，每个文件单独编译成一个 crate

像外部用户一样，只能用 pub 的东西

● 公共代码放 tests/common/mod.rs，不能放 tests/common.rs

tests 下面直接放的 .rs 文件，每个都会被当成一个测试 crate

● 每个测试文件开头写 mod common;

公共代码会被编进每一个测试 crate



Cargo.toml: 测试用的依赖



Cargo.toml

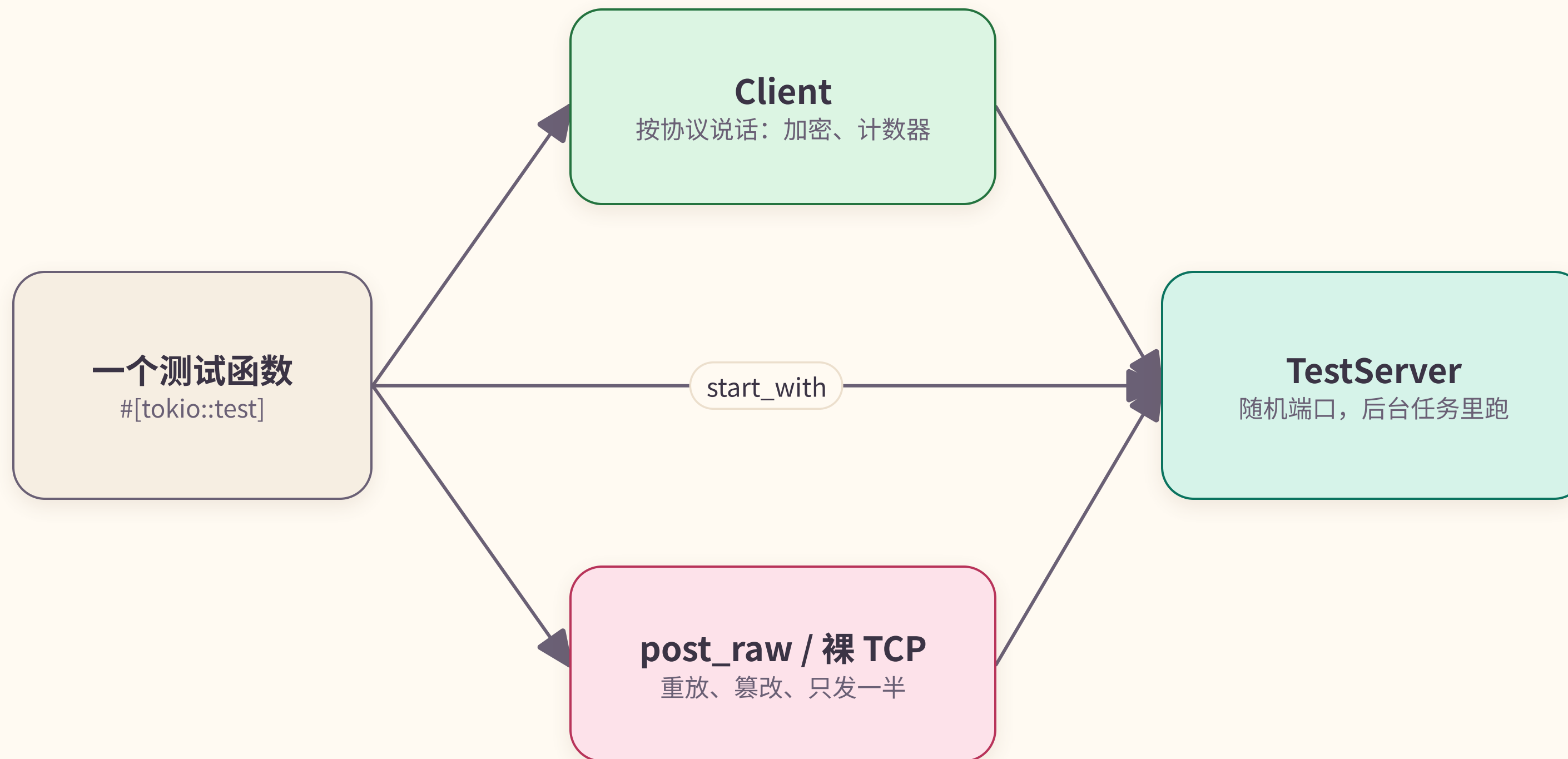
crates/lanshare/

第 13-32 行 • 共 32 行

```
13 axum = "0.8"
14 base64 = "0.22"
15 dirs = "6"
16 hex = "0.4"
17 hyper = { version = "1", features = ["http1", "server"] }
18 hyper-util = { version = "0.1.21", features = ["tokio", "server-graceful"] }
19 if-addr = "0.13"
20 lanshare-proto = { path = "../lanshare-proto" }
21 qrcode = { version = "0.14", default-features = false, features = ["svg"] }
22 rand_core = { version = "0.6", features = ["getrandom"] }
23 serde = { version = "1", features = ["derive"] }
24 serde_json = "1"
25 tokio = { version = "1", features = ["rt-multi-thread", "macros", "net", "fs", "io-util", "sync", "time", "signal"] }
26 tower = { version = "0.5", features = ["util"] }
27 windows-sys = { version = "0.61.2", features = ["Win32_System_SystemInformation", "Win32_Storage_FileSystem"] } 临时
28
29 [dev-dependencies]
30 request = { version = "0.12", default-features = false, features = ["json"] }
31 sha2 = "0.10"
32 tempfile = "3"
```



测试是怎么跑起来的





common/mod.rs: 开头和假桌面

mod.rs crates/lanshare/tests/common/

第 6-25 行 • 共 25 行

```
6 use std::sync::{Arc, Mutex};
7
8 use lanshare::server::{self, AppState, Config, Desktop};
9 use lanshare_proto as proto;
10 use rand_core::{OsRng, RngCore};
11 use serde_json::{Value, json};
12
13 #[derive(Default)]
14 pub struct FakeDesktop {
15     pub calls: Mutex<Vec<(String, PathBuf)>>,
16 }
17
18 impl Desktop for FakeDesktop {
19     fn reveal(&self, path: &Path) {
20         self.calls.lock().unwrap().push(("reveal".into(), path.to_path_buf()));
21     }
22     fn open_folder(&self, path: &Path) {
23         self.calls.lock().unwrap().push(("folder".into(), path.to_path_buf()));
24     }
25 }
```



common/mod.rs: TestServer

mod.rs crates/lanshare/tests/common/

第 27-46 行 • 共 46 行

```
27 pub struct TestServer {
28     pub base: String,
29     pub port: u16,
30     pub state: Arc<AppState>,
31     pub dir: tempfile::TempDir,
32     pub desktop: Arc<FakeDesktop>,
33     pub logs: Arc<Mutex<Vec<String>>>,
34 }
35
36 impl TestServer {
37     pub fn share(&self) -> PathBuf {
38         self.dir.path().join("share")
39     }
40     pub fn key(&self) -> [u8; 32] {
41         self.state.key()
42     }
43     pub fn logs(&self) -> String {
44         self.logs.lock().unwrap().join("\n")
45     }
46 }
```



common/mod.rs: 启动服务端

mod.rs

crates/lanshare/tests/common/

第 59-78 行 • 共 78 行

```
59     config.log = Arc::new(move |line: &str| sink.lock().unwrap().push(line.to_string()));
60     tweak(&mut config);
61     let state = Arc::new(AppState::new(config, port));
62     tokio::spawn(server::serve(listener, state.clone()));
63     TestServer { base: format!("http://127.0.0.1:{port}"), port, state, dir, desktop, logs }
64 }
65
66 pub async fn start() -> TestServer {
67     start_with(Ipv4Addr::LOCALHOST, |_| {}).await
68 }
69
70 pub fn http() -> reqwest::Client {
71     reqwest::Client::builder().no_proxy().build().unwrap()
72 }
73
74 pub fn random_cid() -> [u8; 16] {
75     let mut cid = [0u8; 16];
76     OsRng.fill_bytes(&mut cid);
77     cid
78 }
```



common/mod.rs: 测试客户端

mod.rs crates/lanshare/tests/common/

第 119-138 行 • 共 138 行

```
119         .await
120         .unwrap();
121     (resp.status().as_u16(), resp.bytes().await.unwrap().to_vec())
122 }
123
124 /// 加密发送并解开响应; 状态码不是 200 时返回 Err(状态码)。
125 pub async fn send(&mut self, path: &str, plain: &[u8]) -> Result<Vec<u8>, u16> {
126     let (ctr, body) = self.seal(path, plain);
127     let (status, bytes) = self.post_raw(path, ctr, body).await;
128     if status != 200 {
129         return Err(status);
130     }
131     Ok(proto::open(&self.keys.s2c, ctr, &proto::resp_aad(path), &bytes).expect("响应必须能用 s2c 解开"))
132 }
133
134 pub async fn rpc(&mut self, request: Value) -> Value {
135     let bytes = self.send("/api/rpc", request.to_string().as_bytes()).await.expect("rpc 应返回 200");
136     serde_json::from_slice(&bytes).unwrap()
137 }
138 }
```



common/mod.rs: 两种配对

mod.rs crates/lanshare/tests/common/

第 164–183 行 • 共 183 行

```
164     let pake_key = proto::pake_finish(state, &peer).unwrap();
165     let server_tag = hex::decode(reply["confirm"].as_str().unwrap()).unwrap();
166     if !proto::verify_confirm(&pake_key, "server", &cid, &server_tag) {
167         return Err("wrong_pin".into());
168     }
169     let resp = http()
170         .post(format!("{base}/api/pake/finish"))
171         .json(&json!({ "cid": hex::encode(cid), "confirm": hex::encode(proto::confirm_tag(&pake_key, "client", &cid)) }))
172         .send()
173         .await
174         .unwrap();
175     if resp.status() != 200 {
176         return Err(resp.status().as_u16().to_string());
177     }
178     Ok(Client::from_secret(base, cid, &proto::session_secret_from_pake(&pake_key, &cid)))
179 }
180
181 pub async fn server_id(server: &TestServer) -> String {
182     server.state.server_id().to_string()
183 }
```



common/mod.rs: 分块传输的工具

mod.rs crates/lanshare/tests/common/

第 221-240 行 • 共 240 行

```
221         Some((1, err)) => Err(serde_json::from_slice::<Value>(err).unwrap()["error"].as_str().unwrap().to_string()),
222         _ => panic!("下载响应缺状态字节"),
223     }
224 }
225
226 /// 按列表里的 id 和大小下载整个文件。
227 pub async fn download(&mut self, file_id: &str, size: u64, mtime: f64) -> Result<Vec<u8>, String> {
228     let chunks = (size as usize).div_ceil(proto::CHUNK).max(1);
229     let mut out = Vec::with_capacity(size as usize);
230     for index in 0..chunks {
231         out.extend(self.download_chunk(file_id, index, json!({ "size": size, "mtime": mtime })).await?);
232     }
233     Ok(out)
234 }
235
236 pub async fn find_file(&mut self, name: &str) -> Value {
237     let list = self.rpc(json!({ "op": "list" })).await;
238     list["files"].as_array().unwrap().iter().find(|f| f["name"] == name).cloned().expect("列表里应该有这个文件")
239 }
240 }
```



api.rs: 握手、信道、RPC 的攻击用例

api.rs crates/lanshare/tests/ 从仓库拷贝

第 349-361 行 • 共 361 行

```
349
350 #[tokio::test]
351 async fn logs_never_contain_secrets() {
352     let s = start().await;
353     let mut c = pair_qr(&s).await;
354     c.rpc(json!({ "op": "text_add", "text": "机密文字" })).await;
355     let _ = pair_pin(&s.base, &s.state.pin(), &server_id(&s).await).await;
356     let _ = hello(&s.base, &[9u8; 32], &random_cid()).await;
357     let logs = s.logs();
358     for secret in [hex::encode(s.key()), s.state.pin(), "机密文字".to_string()] {
359         assert!(!logs.contains(&secret), "日志里出现了秘密: {secret}\n{logs}");
360     }
361 }
```



一个时好时坏的测试





改之前和改之后

原来：真的发一个超长正文

```
1 #[tokio::test]
2 async fn oversized_rpc_is_413() {
3     let s = start().await;
4     let mut c = pair_qr(&s).await;
5     let huge = vec![b'x'; proto::MAX_RPC + 100];
6     assert_eq!(c.send("/api/rpc", &huge).await.unwrap_err(), 413);
7 }
```

现在：只发请求头

```
1 let head = format!(
2     "POST /api/rpc HTTP/1.1\r\nHost: 127.0.0.1:{}\r\nContent-Type:
3     application/octet-stream\r\n\r\n
4     X-LS-Sid: {}\r\nX-LS-Ctr: 1\r\nContent-Length: {}\r\n\r\n",
5     s.port,
6     hex::encode(c.cid),
7     proto::MAX_RPC + proto::TAG_LEN + 1
8 );
9 let mut stream = tokio::net::TcpStream::connect(("127.0.0.1",
10 s.port)).await.unwrap();
11 stream.write_all(head.as_bytes()).await.unwrap();
12 let mut buf = vec![0u8; 256];
13 let n = tokio::time::timeout(std::time::Duration::from_secs(3), stream.read(&mut
14 buf)).await.unwrap().unwrap();
15 assert!(String::from_utf8_lossy(&buf[..n]).starts_with("HTTP/1.1 413 "), "声明的长
16 度超过上限，不读正文直接 413");
```



transfer.rs: 完整性、续传、并发

● ● ● transfer.rs crates/lanshare/tests/ 从仓库拷贝 第 154-173 行 • 共 190 行

```
154 let begin = c.rpc(json!({ "op": "upload_begin", "name": "p.bin", "size": data.len() })).await;
155 let id = begin["upload_id"].as_str().unwrap().to_string();
156 // 先按顺序加密好 4 块, 再并发发出去 (浏览器也是这样)
157 let mut sealed = Vec::new();
158 for (i, chunk) in data.chunks(proto::CHUNK).enumerate() {
159     let path = format!("/api/up/{id}/{i}");
160     let (ctr, body) = c.seal(&path, chunk);
161     sealed.push((path, ctr, body));
162 }
163 // 倒序、真正并发地发出去 (每个任务各自持有 HTTP 客户端的克隆)
164 let tasks: Vec<_> = sealed
165     .into_iter()
166     .rev()
167     .map(|(path, ctr, body)| {
168         let (http, base, sid) = (c.http.clone(), c.base.clone(), hex::encode(c.cid));
169         tokio::spawn(async move {
170             http.post(format!("{base}{path}"))
171                 .header("X-LS-Sid", sid)
172                 .header("X-LS-Ctr", ctr.to_string())
173                 .header("Content-Type", "application/octet-stream")
```



wire.rs: 抓包测试

● ● ●

wire.rs crates/lanshare/tests/ 从仓库拷贝

第 112-117 行 • 共 117 行

```
112         assert!(!contains(&tape, secret), "抓包里出现了: {what}");
113     }
114     // 反向自检: 确认这个检查本身是有效的—明文协议字段确实能在录音里找到
115     assert!(contains(&tape, b"/api/hello"), "录音里应该能看到明文的请求行");
116     assert!(contains(&tape, hex::encode(cid).as_bytes()), "会话 id 本来就是明文, 应该能找到");
117 }
```



hardening.rs: 审查发现的回归测试



hardening.rs

crates/lanshare/tests/

从仓库拷贝

第 417-436 行 • 共 451 行

```
417 // 复验 N-5: 只查进度、不传块, 不能让上传一直占着预留的空间
418 #[tokio::test]
419 async fn status_polling_does_not_keep_an_upload_alive() {
420     let s = start_with(Ipv4Addr::LOCALHOST, |c| c.upload_idle = Duration::from_millis(600)).await;
421     let mut c = pair_qr(&s).await;
422     let id = c.rpc(json!({ "op": "upload_begin", "name": "x", "size": 3 })).await["upload_id"].as_str().unwrap().to_string();
423     let mut last = json!(null);
424     for _ in 0..6 {
425         tokio::time::sleep(Duration::from_millis(300)).await;
426         last = c.rpc(json!({ "op": "upload_status", "upload_id": id })).await;
427     }
428     assert_eq!(last["error"], "not_found", "1.8 秒没有新块, 早该过期了");
429 }
430
431 #[tokio::test]
432 #[cfg(windows)]
433 async fn begin_keeps_a_disk_reserve() {
434     let s = start_with(Ipv4Addr::LOCALHOST, |c| c.disk_reserve = 1 << 62).await;
435     let mut c = pair_qr(&s).await;
436     let r = c.rpc(json!({ "op": "upload_begin", "name": "x", "size": 1 })).await;
```



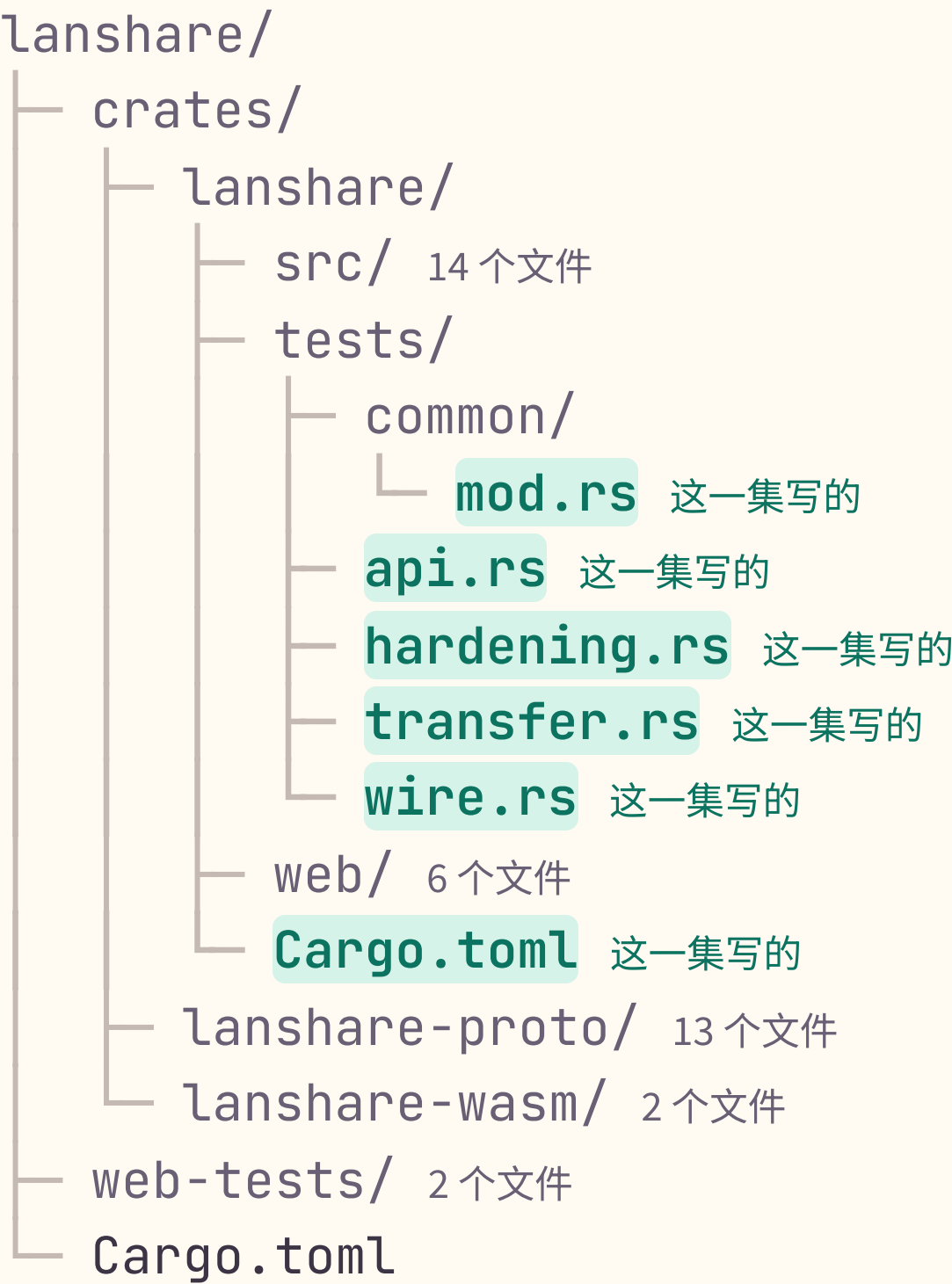
变异测试：把修复故意改坏

故意改坏	抓住它的测试
空闲计时永不到期	idle_connections_are_closed
去掉每会话上传名额	uploads_per_session_are_capped
去掉 Origin 检查	cross_site_origin_is_refused
不检查会话就去读正文	unknown_session_is_rejected_before_the_body_arrives
去掉每 IP 连接上限、本机单独一池、请求头时限、每 IP 正文名额	各自对应的测试
rename_no_replace 改回 fs::rename	第一次没抓住：先查 exists() 把问题掩盖了
去掉 pake_start 里的“已暂停”检查	第一次没抓住：finish 的第二道防线拦住了





测试写好了





跑全部测试

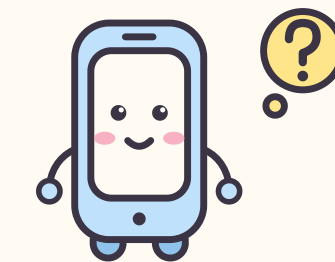
```
Windows PowerShell

PS D:\code\lanshare> cargo test -p lanshare
Updating crates.io index
Locking 40 packages to latest Rust 1.96.0 compatible versions
Adding base64 v0.23.1
Adding bumpalo v3.20.3
..... (省略 14 行, 都是 Adding)
Adding reqwest v0.12.28 (available: v0.13.5)
..... (省略 23 行, 都是 Adding)
Compiling stable_deref_trait v1.2.1
..... (省略 43 行, 都是 Compiling)
Compiling reqwest v0.12.28
Compiling lanshare v2.0.0 (D:\code\lanshare\crates\lanshare)
Finished `test` profile [unoptimized + debuginfo] target(s) in 48.08s
Running unittests src\lib.rs (target\debug\deps\lanshare-86f5b2d3292c0cf0.exe)

running 23 tests
..... (省略 23 行, 全部 ok)

test result: ok. 23 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.01s

Running unittests src\main.rs (target\debug\deps\LanShare-f9c10d633f2e59d9.exe)
```



选择题

公共代码为什么写成 `tests/common/mod.rs`，而不是 `tests/common.rs`？

- A tests 下面直接放的 .rs 文件，每个都会被 cargo 当成一个测试 crate
- C common.rs 不能写 pub 函数

B mod.rs 编译得更快



选择题

公共代码为什么写成 `tests/common/mod.rs`，而不是 `tests/common.rs`？

A `tests` 下面直接放的 `.rs` 文件，每个都会被 `cargo` 当成一个测试 crate

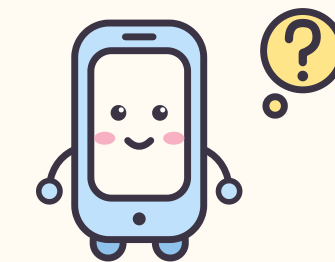
写成 `tests/common.rs`，`cargo` 会把它单独编译、单独跑，输出里多一个“`running 0 tests`”。放进子文件夹，它只是被别的测试文件用 `mod common` 引入的模块。

B `mod.rs` 编译得更快

和编译速度无关。

C `common.rs` 不能写 `pub` 函数

能写，问题在于 `cargo` 会把它当成测试文件。



选择题

wire.rs 最后为什么还要断言“录音里能找到 api 斜杠 hello 和会话号”？

- A 为了确认它们被加密了
- B 防止测试空跑：代理没录到东西、或者搜索写错了，“找不到秘密”就毫无意义
- C 为了测会话号的格式



选择题

wire.rs 最后为什么还要断言“录音里能找到 api 斜杠 hello 和会话号”？

A 为了确认它们被加密了

恰恰相反，它们本来就是明文，断言的是“能找到”。

C 为了测会话号的格式

格式在别的测试里查，这里是自检。

B 防止测试空跑：代理没录到东西、或者搜索写错了，“找不到秘密”就毫无意义

本来就是明文的东西能被找到，才说明代理确实录到了流量、搜索函数确实有效。这样“搜不到秘密”才是可信的证据。



选择题

N-4 的并发测试，第一版用默认的 tokio test，修复前就通过了。为什么？

A 默认是单线程运行时，20 个请求其实是一个接一个处理的

B 20 个请求太少

C 测试写错了断言



选择题

N-4 的并发测试，第一版用默认的 tokio test，修复前就通过了。为什么？

A 默认是单线程运行时，20 个请求其实是一个接一个处理的

单线程运行时里，任务轮流执行，加锁前后不会被别的请求插进来，竞态根本不会出现。换成多线程、8 个工作线程，修复前稳定复现出 8 到 9 次。

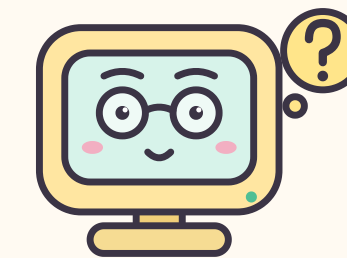
B 20 个请求太少

数量不是问题，问题是它们根本没有同时执行。

C 测试写错了断言

断言没错，是运行时没有制造出并发。

想一想



一个测试在修复之前就已经是绿的。它有没有用？



想一想

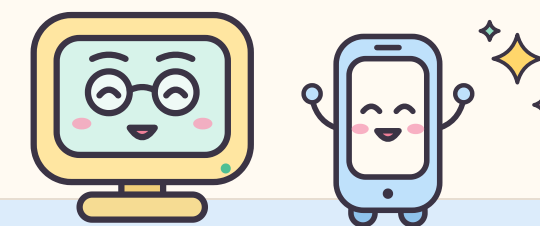
一个测试在修复之前就已经是绿的。它有没有用？

要分情况。如果它是用来证明“修复有效”的，那它没用：修复前就绿，说明它根本没有测到那个问题，比如 N-4 那个单线程的测试。但也有一类测试本来就该一直绿，书里叫它们护栏，比如“正常登录不计入暂停次数”：它防的是修过头，把正常用户也挡住。第一轮写 hardening 时，13 个红，2 个本来就绿，就是这两类的区别。所以写回归测试时，先确认它在修复前是红的；写护栏时，确认它在修复后仍然是绿的。



这一集写了什么

- ✓ 集成测试在 tests 目录，每个文件一个 crate；公共代码放 common/mod.rs
- ✓ 脚手架：端口 0、可调的限额、假桌面、收集日志，服务端在后台任务里真实地跑
- ✓ 测试客户端：send 守规矩，post_raw 当捣蛋鬼
- ✓ 抓包测试录下每一个字节，搜不到秘密；反向自检保证它不是空跑
- ✓ 每条审查发现一个回归测试，变异测试检验测试本身



下一集

全课完结，感谢收看